

SDK & INTEGRATION GUIDE

Inaya Custody SDK

Client-Side Cryptographic Sovereignty for Distributed Applications

```
$ npm install github:Talhawaqas/custody-sdk ethers
```

Private repo — collaborator access + git auth required. Public npm publish pending (see §1.1).

SECTION 01

Overview & Core Philosophy

Encrypt and fragment multi-tenant data structures in under 5 lines of code.

The Inaya SDK gives distributed applications (dApps) client-side data sovereignty: files are encrypted and split into two independent shards entirely in the caller's own runtime, before anything ever leaves the device. The crypto layer is built on `@noble/hashes` and `@noble/ciphers` rather than the browser's `crypto.subtle` — a deliberate choice, since React Native has no native SubtleCrypto implementation, and noble's pure-JS primitives produce byte-identical PBKDF2 keys and AES-GCM ciphertext, so shards stay decryptable across web, Node.js, and mobile alike.

PBKDF2 KEY DERIVATION

AES-GCM-256 ENCRYPTION

DUAL-VECTOR SHARDING

EVM LEDGER ATTESTATION

IPFS SWARM ROUTING

1.1 Installation

The package is not yet on the public npm registry — today it installs directly from the private GitHub repository:

SHELL — INSTALLATION

```
$ npm install github:Talhawaqas/custody-sdk ethers
```

Requirements — Node.js 18+, and ethers (v6) as a peer dependency. Works in two modes: browser (a Web3 wallet such as MetaMask, via `InayaKernel.connectWallet()`), or Node.js/server-side (pass a raw `ethers.Wallet` directly as the connection param instead — no injected provider needed).

Access — This is a private repository. Collaborator access must be granted first, and git authentication (SSH key or a GitHub personal access token) configured locally — `npm install github:... clones the repo` under the hood, using the same auth as any private GitHub install. A public npm publish is planned but has not happened yet; do not assume `"npm install @inaya-network/custody-sdk"` works today.

SECTION 02

Step-by-Step Implementation Workflow

Step 1 — Derive a Local Vault Key

A vault key is derived once, locally, via PBKDF2 (100,000 iterations by default), and can be reused across multiple `disperseAndSlice()` calls rather than re-derived every time.

JAVASCRIPT — VAULT KEY DERIVATION

```
import { InayaKernel } from '@inaya-network/custody-sdk';

// 1. Generate a random 16-byte salt
const passkey = 'user_secure_administrative_passkey';
const salt = InayaKernel.generateSecureSalt(16);

// 2. Derive a 256-bit AES key via PBKDF2 (100,000 iterations)
const vaultKey = await InayaKernel.deriveVaultKey({
  passkey,
  salt,
  iterations: 100000,
  algo: 'HMAC-SHA256', // or HMAC-SHA384 / HMAC-SHA512
});

// vaultKey = { key: Uint8Array(32), salt, iterations, algo }
// key is a raw derived byte array -- NOT a browser CryptoKey --
// which is exactly why this also works in React Native.
```

Step 2 — Encrypt & Slice Into Two Shards

The file is encrypted with AES-GCM-256 using the derived vault key, then the resulting ciphertext is bisected at its exact byte midpoint. Neither shard alone contains a contiguous, decryptable structure.

JAVASCRIPT — SLICING & DISPERSAL

```
const file = fileInput.files[0];
const { filename, shardAlpha, shardBeta } = await InayaKernel.disperseAndSlice({
  file,
  encryptionKey: vaultKey, // the object from Step 1, not a bare key
});

// Pin shardAlpha and shardBeta to IPFS yourself (e.g. via Pinata) --
// the SDK does not pin shards for you.
```

Step 3 — Approve Fee Tokens

Custody's on-chain write charges a per-GB fee in USDT and INAYA, read live from the contract (`usdtFeePerGB / inayaFeePerGB`) — there is no fixed pricing-tier enum. Call `approveFeeTokens()` once per file before anchoring, or `anchorToLedger()` reverts on the internal token transfer.

JAVASCRIPT — FEE APPROVAL

```
const connection = await InayaKernel.connectWallet(); // browser
// -- or, server-side: const connection = { provider: new ethers.Wallet(pk,
// rpcProvider) };

const { usdtFee, inayaFee } = await InayaKernel.approveFeeTokens({
  connection,
  fileSizeBytes: file.size,
});
```

Step 4 — Anchor to the Ledger

Both shard references are written on-chain via `InayaCustody.batchRegisterAssets()` (a single file is submitted as a 1-element batch). This write is permanent by design — the contract has no update or delete function for a registered asset, confirmed by testing every plausible mutation selector live against the deployed contract.

JAVASCRIPT — LEDGER ANCHOR

```
const result = await InayaKernel.anchorToLedger({
  connection,
  fileName: file.name,
  fileSizeBytes: file.size,
  dataShardAlpha: shardAlpha, // the IPFS CID for Shard Alpha
  dataShardBeta: shardBeta,   // the IPFS CID for Shard Beta
  onProgress: (e) => console.log(e.stage), // hashing -> submitting -> confirming -> done
});

console.log(`Anchored: ${result.transactionHash}, assetId: ${result.assetId}`);
```

Write-once by design — `batchRegisterAssets()` is genuinely the only function that writes asset data on-chain. Renaming, moving, or deleting a file is handled entirely off-chain by the Metadata client (Section 05) — not by a contract call.

Step 5 — Retrieve & Reconstruct

Reading a file back is symmetric: look up the asset's shard CIDs on-chain, fetch both shards from IPFS, then decrypt locally with only the original passkey (no vault key object needs to be kept around).

JAVASCRIPT — RETRIEVAL

```
const { name, owner, dataUrl } = await InayaKernel.retrieveAndReconstruct({
  connection,
  assetId: result.assetId, // or pass fileHash directly
  passkey,
  onProgress: (e) => console.log(e.stage), // reading-chain -> fetching-shards -> decrypting
});

// dataUrl is a data: URL -- render it or trigger a download directly.
```

SECTION 03

Staking

InayaKernel.Staking wraps the deployed InayaStaking contract, with tiered lock periods and automatic token-approval handling before staking (INAYA transfers carry a fixed micro-fee the caller doesn't need to compute manually).

JAVASCRIPT — STAKE / UNSTAKE / CLAIM

```
await InayaKernel.Staking.stake({ connection, amount, lockPeriodDays: 30 }); // 0, 30, or 90

// withdraw() and claimReward() are two separate on-chain actions --
// there is no combined "unstake and pay out" function.
await InayaKernel.Staking.unstake({ connection, amount });
await InayaKernel.Staking.claimReward({ connection });

const earned = await InayaKernel.Staking.calculateReward({ connection, address });
const staked = await InayaKernel.Staking.getStakedBalance({ connection, address });
```

SECTION 04

Monetization — the Payments Client (No-Wallet Card Flow)

For customers who'd rather pay by card than hold a wallet, `InayaKernel.Payments` is a typed client for card-checkout backend routes you deploy yourself. It contains zero secrets — no Stripe key, no treasury private key — it only calls `fetch()` against your own `/api/*` routes.

Tier	Behavior
Corporate Reserve	Enterprise annual subscription — fixed allocation, billed in USDT with maintenance settled in INAYA.
Pay-As-You-Go (PAYG)	Retail metered billing per upload, settled directly on-chain against the PAYG contract.
Egress Checkout	Pay-per-retrieval unlock for a specific file, checked via <code>getEgressUnlockStatus()</code> before prompting for a passkey.

JAVASCRIPT — CARD CHECKOUT

```
const { checkoutUrl } = await InayaKernel.Payments.startCorporateReserveCheckout({
  tier: 'annual-standard',
});
window.location.href = checkoutUrl; // navigate yourself -- the SDK never redirects for you

// Retail / per-upload:
const payg = await InayaKernel.Payments.startPaygCheckout({
  filename, sizeBytes, cidAlpha, cidBeta, fileHash,
});
```

The names "Scale Matrix", "Established Swarm", "Institutional Node", and a class called `EnterpriseGateway` do not exist anywhere in this codebase — if an earlier version of this guide mentioned them, they were incorrect and have been replaced above with the real tier names used throughout the dApp and backend.

SECTION 05

Off-Chain Metadata Layer

Because Custody's on-chain write is permanent, `InayaKernel.Metadata` fills the gap with a server-backed layer for everything the contract itself can't do: renaming, moving, deleting, restoring, virtual folders, and sharing. Every mutating call is authenticated by a wallet signature, not a bare address, and the reference backend cross-checks the signer against the real on-chain owner before allowing any change.

JAVASCRIPT — METADATA OPERATIONS

```
await InayaKernel.Metadata.renameFile({ connection, fileHash, newName });
await InayaKernel.Metadata.moveFile({ connection, fileHash, folderId });
await InayaKernel.Metadata.deleteFile({ connection, fileHash }); // soft delete
await InayaKernel.Metadata.restoreFile({ connection, fileHash });

await InayaKernel.Metadata.createFolder({ connection, name });
await InayaKernel.Metadata.shareFile({ connection, fileHash, withAddress });
```


SECTION 06

Error Handling, Progress & Retries

Every SDK error is one of five typed classes, so callers can branch on what actually went wrong instead of parsing message strings:

- `InayaValidationError` — bad input caught before any network call.
- `InayaWalletError` — no provider, user rejected a signature, etc.
- `InayaContractError` — an on-chain call reverted.
- `InayaNetworkError` — an RPC, IPFS gateway, or backend fetch failed.
- `InayaError` — the shared base class for instanceof checks.

Long-running calls (`anchorToLedger`, `retrieveAndReconstruct`, `Staking.stake`, `approveFeeTokens`) accept an `onProgress` callback and also emit matching events on the shared `InayaKernel.events` emitter — use whichever pattern fits. Genuinely transient failures (IPFS gateway timeouts, read-only RPC calls) retry automatically with backoff; contract reverts and wallet rejections never do, since retrying those can't help and may actively confuse the user.

JAVASCRIPT — SUBSCRIBING TO EVENTS

```
InayaKernel.events.on('anchor:progress', (e) => console.log(e.stage));  
InayaKernel.events.on('error', (e) => console.error(e.operation, e.error));
```

SECTION 07

API Quick Reference

generateSecureSalt(bytes=16)

Cryptographically random salt via `crypto.getRandomValues`.

deriveVaultKey(params)

PBKDF2-derives a 256-bit AES key once for reuse. Returns { key: Uint8Array, salt, iterations, algo }.

disperseAndSlice(params)

AES-GCM-256 encrypts a file, bisects at the byte midpoint into { filename, shardAlpha, shardBeta }.

reconstructAndDecrypt(params)

Rejoins shards and re-derives the key from a passkey to decrypt back to a data: URL.

connectWallet()

Browser-only EIP-1193 wallet connect (MetaMask, etc.).

approveFeeTokens(params)

Reads live per-GB USDT/INAYA fees and approves both tokens for spend.

anchorToLedger(params)

Registers a file's shard CIDs on-chain via `batchRegisterAssets` (write-once).

retrieveAndReconstruct(params)

Reads on-chain shard CIDs, fetches from IPFS, decrypts locally.

Staking.stake / unstake / claimReward

InayaStaking contract wrapper — tiered lockup of 0, 30, or 90 days.

Staking.calculateReward / getStakedBalance

Read-only staking balance/reward lookups.

Payments.startCorporateReserveCheckout

Card checkout for the enterprise annual plan.

Payments.startPaygCheckout / startEgressCheckout

Card checkout for a single retail upload / a retrieval unlock.

Payments.whoAmI / getCorporatePlanStatus

Reads the current card customer's identity / plan status.

Metadata.renameFile / moveFile / deleteFile / restoreFile

Off-chain file mutations, signature-authenticated.

Metadata.createFolder / shareFile / listSharedWithMe

Virtual folders and sharing, signature-authenticated.

events

Shared `InayaEventEmitter` for lifecycle events (.on / .off / .once).

errors.*

Five typed error classes for instanceof-based branching.

Security note — Passkeys and derived vault keys should never be transmitted off-device or logged. Store only the salt and resulting content identifiers (CIDs) in application state.

"Every file, sliced and sovereign — encrypted where it starts, never whole where it travels."

@inaya-network/custody-sdk • v1.0.4-beta • BNB Chain Testnet • Corrected against source, August 2, 2026